

A Beginner's Guide to Feature Engineering in Machine Learning

 medium.com/@pranidhi_M/a-beginners-guide-to-feature-engineering-in-machine-learning-e03857063628

Pranidhi M

May 17, 2025

Member-only story

INTRODUCTION:

Feature engineering is the process of transforming raw data into meaningful features that improve the performance of machine learning models. This involves selecting, modifying, or creating new features to help algorithms for better understand the patterns in the data.

WHY IS FEATURE ENGINEERING CRITICAL?

Feature engineering plays a crucial role in refining model accuracy by altering the raw data into informative inputs that better capture the underlying patterns. Well-designed features can significantly boost a model's predictive power by highlighting relationships or trends that may not be obvious in the original data. By incorporating the domain knowledge with statistical techniques, data scientists can create new features , such as ratios, differences, or time-based aggregations , that allow models to learn more effectively from the available information. This not only enhances accuracy but also reduces the dependency on complex algorithms to uncover insights buried in noisy datasets.

Furthermore, we can do it by improving the performance. Effective feature engineering helps reduce overfitting by focusing the model on meaningful patterns rather than irrelevant noise. By applying techniques such as imputation for missing values or normalisation to address scale differences, data is made cleaner and more consistent for modelling. In addition, well-structured and interpretable features make model outputs easier to explain to stakeholders, which is crucial for trust and transparency, especially in regulated industries. Ultimately, quality features bridge the gap between raw data and actionable, reliable predictions.

WHERE DOES FEATURE ENGINEERING FIT IN THE ML PIPELINES?

Feature Engineering is a critical step in the machine learning pipeline, positioned between data collection and preprocessing and model training. The primary step is Exploratory Data Analysis (EDA), which helps to understand the data's distribution and uncover underlying patterns or anomalies, which inform the next steps. Following this, feature selection involves choosing the most relevant features and eliminating those that contribute little or no predictive value. Feature transformation, such as scaling or encoding, prepares the data by standardising it to improve

model performance. Additionally, feature extraction techniques like Principal Component Analysis (PCA) are used to reduce the dimensionality of the dataset while retaining the most important information, making the data more manageable and efficient for model training.

In the case of the Sentiment_Analysis.ipynb notebook, this is applied using several methods to enhance the models' ability to understand and process the data. Text data is converted into numerical features using Term Frequency -Inverse Document Frequency (TF-IDF), which captures the importance of words within the context of the corpus. Sentiment labels are encoded using LabelEncoder, transforming categorical labels into a format suitable for machine learning models. The features are then scaled with StandardScaler to ensure that all variables contribute equally to the model. Lastly, PCA is applied to reduce the dimensionality of the feature space, making the model efficient and less prone to overfitting by focusing on the most informative components of the data.

This with label encoder

```
... 3530abcc5 http://twitpic.com/4woj2 I like it... positive
...
3529 e5f0e6ef4b its at 3 am, im very tired but i can't sleep ... negative
3530 416863ce47 All alone in this old house again. Thanks for... positive
3531 6332da480c I know what you mean. My little dog is sinkin... negative
3532 df1baec676 _sutra what is your next youtube video gonna b... positive
3533 469e15c5a8 http://twitpic.com/4woj2 - omgssh ang cute n... positive

sentiment_encoded
0 1
1 2
2 0
3 2
4 2
...
3529 0
3530 2
3531 0
3532 2
3533 2

[3534 rows x 4 columns]
```

```
print(encoder.classes_) # Shows the original category mapping
df.to_csv("encoded_sentiment.csv", index=False)
```

```
['negative' 'neutral' 'positive']
```

WHAT ARE FEATURES?

Features are individual measurable properties or characteristics of the data that are used as inputs for ML models. They represent the variables that the model uses to make predictions or classifications. They can take various forms, such as numerical features, which represent

quantitative data that can be continuous or discrete, like age or temperature. Categorical features represent qualitative data, such as gender or product colour, while text features contain raw textual data that often requires preprocessing, like product reviews or tweets. Time-series features are indexed by time, such as stock prices or daily temperature readings.

In the `Sentiment_Analysis.ipynb` notebook, feature engineering is applied to raw text data to make it suitable for machine learning models. The primary feature is the text data to make it suitable for machine learning models. The primary feature is the text column, which is processed using TF-IDF (Term Frequency — Inverse Document Frequency) vectorisation, transforming the raw text into a numerical feature. new feature, “`sentence_length`”, is derived from the text as a numerical feature, representing the length of each sentence. Principal Components Analysis (PCA) is applied to further optimise the model to reduce the feature dimension. This demonstrates how raw text data can be effectively transformed into structured features for models like Support Vector Machines (SVM), making them more suitable for predictive tasks.

TYPES OF FEATURE ENGINEERING TECHNIQUES:

a) Handling Missing Values:

Missing data can significantly distort analysis and model training by introducing bias or reducing the overall quality of insights. In handling this, common strategies include imputations, where missing values are filled using statistical measures such as the mean, median, or mode, allowing the dataset to remain intact while minimising information loss. Another approach is deletion, which involves removing rows or columns containing missing values when the proportion of missing data is small and unlikely to impact overall patterns, ensuring cleaner inputs for analysis or machine learning models.

b) Encoding Categorical Variables:

Machine Learning models require numeric input, so encoding techniques are essential for transforming categorical data into a usable format. **One-Hot Encoding** creates separate binary columns for each category, which is ideal for non-ordinal data where no inherent order exists among categories. In contrast, **Label Encoding** assigns a unique numeric label to each category, making it more suitable for ordinal data where the categories have meaningful order, allowing models to leverage the relative ranking.

Use **One-Hot Encoding** when dealing with **nominal (non-ordinal) Categorical data**, such as colours, countries, or product types, where there is no inherent ranking or order among the categories — this prevents the model from mistakenly interpreting any ordinal relationship. Use **Label Encoding** when working with **Ordinal data**, like education levels such as high school, bachelor's or master's or rating scales, for instance, low, medium or high, where the numeric labels can reflect and preserve the natural order, allowing the model to make use of the ranking information effectively.

Ordinal Data is categorical data with a meaningful order or **ranking** among the categories, but the intervals between them are not necessarily equal. For example, include education levels (ex, High School < bachelor's < Master's), Customer Satisfaction (ex, Poor < Fair < Good < Excellent), or size labels (ex: Small < Medium < Large). The order matters, but you can't qualify the exact difference between the ranks.

Non-ordinal data, also known as **nominal data**, consists of categories that have no inherent order or ranking. Examples include gender, eye colour, types of cuisine or product IDs. Each Category is just a label without any implied meaning, hierarchy or relationship.

c) Scaling and Normalisation:

Scaling is a preprocessing technique that ensures all features contribute equally to model training by standardising their ranges, which is particularly crucial for algorithms that rely on distance calculations, such as K-Nearest Neighbours (KNN), Support Vector Machines (SVM), and gradient descent-based models like linear or logistic regression. Without scaling, features with larger numerical ranges can disproportionately influence the model's performance, leading to biased results or slow convergence. Techniques like Min-Max Scaling (Range between 0 and 1) and Standardisation (which transforms features to have a mean of 0 and a standard deviation of 1) are commonly used to normalise the data, ensuring all features are treated on a comparable scale during model training and improving the model's accuracy and stability.

d) Creating New Features:

This involves creating new features from existing data to enhance model performance and capture hidden patterns. This can include combining features, like calculating Body Mass Index (BMI) from weight and height, which provides more predictive power than the raw inputs alone. Date and Time decomposition is another technique, where components like year, month, day, or day of the week are extracted from a timestamp to capture temporal trends or seasonality. Moreover, leveraging domain Knowledge allows the creation of meaningful, context-aware features tailored to the specific business problem, which can significantly improve the model's ability to learn relevant relationships in the data.

e) Binning / Bucketing:

Binning, also known as Bucketing, is a technique that converts continuous variables into discrete intervals or categories, which can help capture non-linear relationships and reduce the impact of outliers. By grouping numerical values into bins, models can better identify patterns that may not be apparent in raw continuous data. Binning can also simplify models, enhance interpretability, and improve performance in certain algorithms, especially decision trees, by allowing them to focus on meaningful value ranges rather than individual numeric differences.

f) Text Feature Engineering:

Text data must be converted into a numeric format before it can be used in machine learning models, and several techniques are commonly employed for this purpose. The Bag of Words (BoW) model represents text by counting the occurrences of each word in a document, creating a sparse matrix where each column corresponds to a unique word from the vocabulary. While simple, BoW ignores word order and context. TF-IDF (**Term Frequency–Inverse Document Frequency**) improves on this by weighting word frequency in a document against how commonly the word appears across all documents, emphasising more informative and distinctive words. For capturing deeper semantic meaning and word relationships, word embeddings such as Word2Vec, GloVe, or FastText transform words into dense, continuous vector representations, where similar words are placed closer in a high-dimensional space. These embeddings preserve context and meaning, making them especially powerful for natural language processing tasks.

EVALUATING FEATURES IMPACT:

Evaluating feature impact is a critical step in understanding how each input variable influences a machine learning model's predictions. This process helps in identifying the most informative features, removing irrelevant or redundant ones, and improving both model performance and interpretability. Common techniques include feature importance scores provided by tree-based models like Random Forest and XGBoost, which rank features based on how much they contribute to reducing impurity or improving splits. Permutation importance is another model-agnostic method that measures the decrease in model accuracy when a feature's values are randomly shuffled — if the model's performance drops significantly, the feature is considered important. These approaches offer insights into which features are driving predictions and which may be safely excluded or combined.

Furthermore, correlation analysis can be used to examine the linear relationship between features and the target variable, helping detect multicollinearity or highly correlated predictors that may distort model training. For more complex or black-box models, SHapley Additive exPlanations (SHAP) and LIME (Local Interpretable Modelagnostic Explanations) provide powerful tools to interpret feature impact on individual predictions by attributing contribution scores to each feature. By evaluating feature impact systematically, data scientists can refine feature selection, reduce model complexity and build more transparent, robust, and generalizable models.

COMMON PITFALLS:

When preparing features for machine learning models, several common pitfalls can negatively affect performance and generalizability. One major issue is overfitting due to irrelevant or noisy features, where the model learns patterns that are specific to the training data but do not generalise well to new, unseen data. This often happens when too many features are included without evaluating their predictive power or relevance, leading to increased complexity and poor model performance on validation sets. Carefully selecting and engineering meaningful features, along with regularisation techniques and cross-validation, can help mitigate this risk.

Another critical pitfall is data leakage, which occurs when information from outside the training dataset — or from the target variable itself — is inadvertently used in the model. This can result in unrealistically high accuracy during training and testing, but leads to failure in real-world deployment. Similarly, multicollinearity, where two or more features are highly correlated, can distort model coefficients and reduce interpretability, particularly in linear models. Detecting and addressing multicollinearity using correlation matrices or the Variance Inflation Factor (VIF) analysis is important for building stable and reliable models. Avoiding these pitfalls is essential for creating robust, accurate, and trustworthy machine learning solutions.

BEST PRACTICES:

Adopting best practices in feature engineering ensures reproducibility, efficiency, and robustness of machine Learning workflows. One key practice is to **automate the feature engineering process using pipelines**, such as **sklearn. Pipelines**, which allow for sequential chaining of preprocessing steps, feature transformations, and model training. This not only reduces manual errors and redundancy but also ensures consistency between the training and inference phases. Pipelines help encapsulate transformations in a structured manner, making the code cleaner, modular, and easier to maintain or deploy.

Another essential practice is to **always evaluate the model on separate validation and test sets** to avoid overfitting and obtain a realistic estimate of performance on unseen data. This helps ensure that feature transformations and model tuning do not inadvertently leak information from the test set, preserving the integrity of evaluation. Furthermore, it's crucial to document all feature transformations — including scaling, encoding and feature creation steps — to ensure transparency and reproducibility. Clear documentation aids collaboration, eases debugging, and is especially valuable when models are handed off to other teams or deployed in production environments.

CONCLUSION:

In conclusion, effective feature engineering is foundational to building high-performing machine learning models. Good features capture the underlying patterns in data, enhance model accuracy, and often make the difference between a mediocre and a highly predictive model. While advanced algorithms can help, they cannot compensate for poorly chosen or irrelevant features — **no amount of modelling can fix bad data**. That's why careful attention to data quality, relevance, and representation is critical in any machine learning pipeline.

Moreover, successful feature engineering requires both **systematic experimentation** and **domain knowledge**. Understanding the context behind the data enables the creation of insightful features that models can learn from more effectively. Practitioners are encouraged to iterate, test, and validate different approaches while staying grounded in the business or real-world problem at hand. Ultimately, the combination of thoughtful engineering and domain-driven insights leads to more robust, interpretable, and impactful models.

For more, follow me at [LinkedIn](#) and on [GitHub](#).